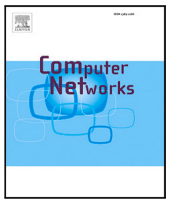
Computer Networks 257 (2025) 110973

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet



EAPT: An encrypted traffic classification model via adversarial pre-trained transformers

Mingming Zhan [a], Jin Yang [a,b,*], Dongqing Jia [a], Geyuan Fu [a]

[a] School of Cyber Science and Engineering, Sichuan University, Chengdu, 610207, Sichuan, China
[b] Key Laboratory of Data Protection and Intelligent Management of the Ministry of Education, Chengdu, 610207, Sichuan, China

ARTICLE INFO

Keywords:
Encrypted traffic classification
Pre-train
Transformer
Replaced BURST detection

ABSTRACT

Encrypted traffic classification plays a critical role in network traffic management and optimization, as it helps identify and differentiate between various types of traffic, thereby enhancing the quality and efficiency of network services. However, with the continuous evolution of traffic encryption and network applications, a large and diverse volume of encrypted traffic has emerged, presenting challenges for traditional feature extraction-based methods in identifying encrypted traffic effectively. This paper introduces an encrypted traffic classification model via adversarial pre-trained transformers-EAPT. The model utilizes the SentencePiece to tokenize encrypted traffic data, effectively addressing the issue of coarse tokenization granularity, thereby ensuring that the tokenization results more accurately reflect the characteristics of the encrypted traffic. During the pre-training phase, the EAPT employs a disentangled attention mechanism and incorporates a pre-training task similar to generative adversarial networks called Replaced BURST Detection. This approach not only enhances the model's ability to understand contextual information but also accelerates the pre-training process. Additionally, this method minimizes model parameters, thus improving the model's generalization capability. Experimental results show that EAPT can efficiently learn traffic features from small-scale unlabeled datasets and demonstrate excellent performance across multiple datasets with a relatively small number of model parameters.

1. Introduction

With the rapid development and widespread use of the Internet, the continuous growth of network traffic has become an inevitable trend [1]. One of the key challenges in current network management and optimization is how to effectively analyze and utilize this traffic. With the growing demand for privacy protection, protocols and applications increasingly employ encryption techniques for communication. As a result, the proportion of encrypted traffic in communication networks has significantly increased. However, while encryption enhances confidentiality, it also poses difficulties for network supervision, as it makes capturing attacks more challenging [2]. With the rapid growth of network traffic, classifying encrypted network traffic effectively and accurately is a challenging problem [3].

Currently, various encryption mechanisms are applied in communication networks [4], such as VPN, VoIP, P2P, SSL, etc. These encryption algorithms make the classification of encrypted traffic more challenging, posing a significant challenge for network traffic identification [5]. Network traffic identification [6] is a technique for determining the protocol or application to which network traffic belongs based on the content or behavioral characteristics of network packets. Traditional methods for network traffic identification include three types: port-based mapping techniques [7], payload analysis-based methods [8], and behavior-based approaches [9]. Port-based mapping methods are simple to implement, identifying traffic by comparing packet port fields with specified port numbers. However, they also have limitations, such as inconsistencies between port numbers and applications, the use of dynamic port numbers, and issues with certain applications hiding or misusing port numbers, resulting in poor performance in traffic identification. Deep Packet Inspection (DPI) [10] is a representative method based on payload analysis. It achieves traffic identification through payload field matching with a rule library. While this method offers high accuracy, it requires continuous updates to the rule library, resulting in significant system overhead. Moreover, it may also involve issues related to user privacy and data security. Behavior-based methods classify and identify network traffic based on statistical features of the traffic. However, this approach suffers from high system resource and time consumption and poor real-time performance [11].

Neural networks possess strong self-learning capabilities, allowing them to learn higher-level feature representations from raw data. At the

* Corresponding author.
E-mail address: yangjin66@scu.edu.cn (J. Yang).

https://doi.org/10.1016/j.comnet.2024.110973
Received 17 July 2024; Received in revised form 1 November 2024; Accepted 1 December 2024
Available online 9 December 2024

same time, networks handle a large amount of traffic and continuously generate new traffic. Data-driven deep learning methods are well-suited for studying issues related to network traffic [12]. Consequently, deep learning-based encrypted traffic identification has become a widely researched direction. By training models using abundant encrypted traffic data, deep neural networks can automatically learn the characteristics and behavior patterns of the traffic, enabling accurate identification of encrypted traffic in real-time scenarios [13]. However, deep learning-based encrypted traffic identification methods also have limitations. Due to the complexity and privacy of encryption protocols, models find it challenging to learn effective features from raw traffic without preprocessing [14]. Additionally, because encryption protocols are continuously updated and evolving, the model's recognition capability tends to lag, making it inefficient in handling ever-changing encrypted traffic identification tasks.

To address the issues of insufficient labeled data and model lag, we propose an encrypted traffic classification model achieved through adversarial pre-training transformers—EAPT. It can learn features from unlabeled encrypted traffic to perform traffic classification tasks. Specifically, the model consists of three stages. In the data processing stage, encrypted traffic is preprocessed and tokenized. During the pre-training stage, we use a disentangled attention mechanism for encoding and propose a new pre-training task for encrypted traffic, which consists of two parts: a BURST Generation Model (BGM) that learns the relationships between datagrams within a BURST and generates fake datagrams for replacement, and a BURST Discrimination Model (BDM) that predicts whether each datagram in the input has been replaced by a sample from the generation model. Finally, a small dataset is employed in the fine-tuning stage to adjust parameters for various encrypted traffic classification tasks.

In summary, the main contributions of this paper are as follows:

1. We propose a method for tokenizing encrypted traffic data based on SentencePiece(SP). By using SP to tokenize encrypted traffic data, we address the issue of large tokenization granularity in encrypted traffic data, allowing the tokenization results to better reflect the characteristics of encrypted traffic. This enables the model to better understand the contextual relationships in the encrypted traffic data.
2. During the pre-training stage, we use a disentangled attention mechanism for encoding to mitigate long-distance dependency issues and propose a more effective pre-training task for encrypted traffic: the Replaced BURST Detection task. This task comprises two transformer encoders similar to generative adversarial networks, which accelerates the pre-training process while reducing the number of model parameters and improving the model's generalization ability.
3. We conduct encrypted traffic classification experiments on four datasets and compare our model with other advanced models. The results show that our model achieves a high level of accuracy, recall, and F1-score. Additionally, compared to previous models, the scale of the pre-training dataset required by our model decreases by 86.7%, the number of model parameters decreases by 80%, and the size of the pre-trained model decreases by 83.28%.

The rest of this paper is organized as follows. Section 2 discusses related work. Section 3 detailed introduces our proposed EAPT. Then, in Section 4, we present the experimental results on different datasets, compare them with other methods, and analyze the results. Finally, in Section 5, we provide our conclusions and future work.

2. Related works

2.1. ML-based encrypted traffic classification

Machine learning-based encrypted traffic classification techniques use manually designed features for classification [15], typically leveraging statistical information of the traffic. These features often result in high identification rates for traffic classification tasks. Kotpalliwar et al. [16] used Support Vector Machines (SVMs) for multi-class traffic classification on the "10% KDD Cup 99" dataset, achieving Validation Accuracy and Classification Accuracy of 89.95% and 99.99%, respectively. However, this study did not evaluate the recall rate. Chandrasekhar et al. [17] combined Fuzzy-C-means clustering, Artificial Neural Networks (ANN), and SVM methods, achieving good results across multiple datasets, but their method was ineffective in detecting anomalies in actual networks. Lashkari et al. [18] used manually designed time-related features and the Random Forest (RF) algorithm to conduct experiments on Tor-non Tor data, achieving high real-time performance, but the accuracy was not high. Li et al. [19] proposed a user behavior classification method, FOAP, which filters out traffic segments unrelated to the application of interest by designing a concept of structural similarity between flows. It then identifies user actions on specific UI components by inferring entry point methods associated with those components. FOAP achieved F1 scores of 0.911 for application recognition and 0.885 for user behavior recognition. Fu et al. [20] proposed a malicious traffic detection system based on real-time unsupervised machine learning (ML), which leverages a compact memory graph constructed from traffic patterns to identify unknown patterns of encrypted malicious traffic. This approach achieves a high level of accuracy.

In recent years, machine learning methods have made significant progress in encrypted traffic classification and anomaly detection, overcoming the limitations of traditional methods. However, the performance of machine learning methods heavily depends on feature design and the choice of classification algorithms. Manually designing features is a tedious and time-consuming task [21]. Furthermore, with the proliferation of the internet and the increasing complexity of network traffic, the characteristics of abnormal traffic have become diverse and variable. When new traffic types emerge, experts need to redesign feature engineering, significantly limiting the generalizability of these methods [22].

2.2. DL-based encrypted traffic classification

Deep learning-based encrypted traffic classification techniques automate feature extraction through training [23], thereby avoiding the need for manual feature design by experts. This method allows the model to learn effective features from the data, leading to better classification performance [24]. Common deep learning methods include CNN, LSTM, and RNN. Yuan et al. [25] employed a neural network model composed of CNNs, RNNs, and fully connected layers, where RNNs can learn longer historical features, effectively reducing the false positive rate and outperforming random forests in terms of generalization.

Cui et al. [26] proposed an encrypted traffic classification method using Capsule Networks (CapsNet) based on session packets. CapsNet can learn complex spatial relationships in input data and was validated on the ISCX VPN-nonVPN dataset, showing high accuracy in encrypted traffic classification compared to traditional methods. Shapira et al. [27] proposed a method called FlowPic, which converts basic traffic data into intuitive images and then uses well-known image classification deep learning techniques, such as CNN, to identify traffic categories. This approach has demonstrated good performance across multiple datasets.

Sirinam et al. [28] proposed a new website fingerprinting method called Deep Fingerprinting (DF), based on CNN and deep learning techniques, capable of identifying websites visited by users from encrypted network traffic. Experiments showed that DF achieved 98.3% accuracy on undefended anonymous network (Tor) traffic and could effectively identify traffic on defended Tor traffic as well. Liu et al. [29] proposed a method combining machine learning algorithms with RNN, called Flow Sequence Network (FS-Net), which learns representative features from

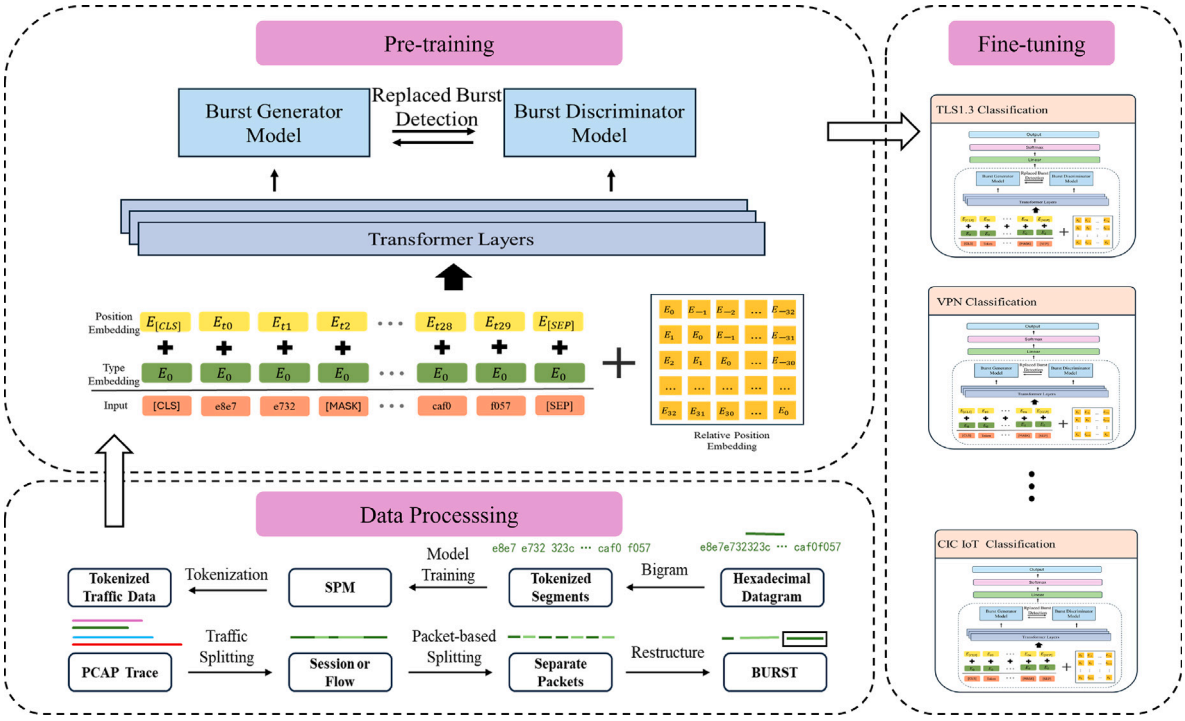


Fig. 1. The framework of EAPT.

raw traffic and performs classification within a unified framework. FS-Net achieved a recall rate of 99.14% and a false positive rate of 0.05% using a real-world dataset that encompasses 18 applications.

Shen et al. [30] in 2021 proposed a decentralized application recognition method based on Graph Neural Networks (GNNs), named Graph-DApp. This method converts data packets generated by interactions between clients and servers into a graph form and uses a GNN-based classifier for detection, showing good performance in classification accuracy and training time. Additionally, there has been a trend of transferring pre-trained models [31,32] to encrypted traffic classification tasks. He et al. [33] first proposed the use of pre-trained models for encrypted traffic classification, introducing the PERT model, which achieved 93.23% performance on the ISCX-VPN-Service dataset in terms of F1-score. Lin et al. [14] further proposed the ET-BERT pre-trained model, which showed good results across multiple datasets.

Currently, deep learning-based encrypted traffic classification tasks have achieved promising results. However, most neural networks rely on large amounts of labeled data to perform well. In the context of network traffic, there is an imbalance between normal and abnormal traffic data [34]. Additionally, as networks continue to evolve, new traffic types emerge, making it difficult to obtain high-quality, large-scale labeled data [35]. The emergence of pre-trained models [36] has partially addressed this issue, but existing pre-training methods still require large amounts of training data and incur high training costs [37].

3. Our method

The structure of our proposed EAPT is shown in Fig. 1, which is mainly divided into three stages: Data Processing, Pre-training, and Fine-tuning.

3.1. Data processing

To process the raw encrypted traffic data into sequences that the EAPT model can understand, we need to handle the captured packets in PCAP files. A session flow in the traffic packets contains multiple packets from both communication parties over a period of time. These multiple packets in the same direction are typically referred to as BURSTs. For our current task, we need to apply a tokenization algorithm to the individual traffic packets within a BURST to obtain the constituent elements of each packet. Previous tokenization algorithms have had issues with tokenization granularity. For example, the structure of traffic packets in the VMESS protocol, as shown in Table 1, varies in data size from 4 bits to 16 bytes. However, past algorithms like bigram divide words into fixed sizes, which can overlook data bytes larger than the fixed size. This omission can negatively impact the model's recognition accuracy, highlighting the need for a better tokenization algorithm.

Table 1
VMess protocol traffic packet structure.

Field name	Length
Version Ver	1 byte
Data Encryption IV	16 bytes
Data Encryption Key	16 bytes
Response Auth V	16 bytes
Options Opt	1 byte
Padding P	4 digit
Encryption Method Sec	1 byte

The data processing stage of our approach follows a detailed workflow as outlined below: Firstly, we classify the captured PCAP files based on sessions. Typically, a session flow comprises multiple data packets exchanged between two communication parties over a period, akin to a dialogue in natural language. Consequently, each packet is treated as a sentence and tokenized using the vocabulary generated by the SentencePiece model. Since tokenizers generally accept only string inputs, we first perform hexadecimal encoding on the complete packets. Following this, we apply bigram processing to the hexadecimal-encoded data, treating adjacent pairs of bytes as a single unit. This approach addresses the semantic diversity and uncertain granularity of traffic packet data, which is similar to processing Chinese characters. As shown in the table, the granularity of traffic data can vary from 4 bits to 16 bytes, making it unsuitable for direct use

with tokenization algorithms like WordPiece or BPE. Inspired by Chinese tokenization methods, we adopt the Bigram tokenization method. However, directly applying methods like WordPiece to bigrammed hexadecimal data could result in excessively fine granularity, which may adversely affect model recognition. Hence, we opt for the SentencePiece tokenization method. SentencePiece treats a sentence as a whole and uses the Unigram algorithm to construct the vocabulary, balancing fine and coarse granularity. If a segment of traffic data appears frequently as a whole, SentencePiece can consider it as a single token, aligning with our traffic data tokenization needs. The Unigram algorithm we employ allows us to define the vocabulary size. Starting from a large vocabulary, it progressively removes tokens until the preset size is met. This approach enables us to predefine the required vocabulary size, avoiding excessive computational overhead during model training. The initial vocabulary includes all tokens output by the pre-tokenizer and all high-frequency substrings. The criterion for removing tokens each time is to minimize the increase in loss. The loss during training is calculated using the following Eq. (1):

$$Loss = -\sum_{i=1}^{\mathbb{N}} \log\Big(\sum_{x \epsilon S(x_i)} p(x)\Big). \tag{1}$$

In Eq. (1), x represents a word in a given dataset. We denote all the words as $x_1; x_2; \dots x_n$. For each word x_i, the set of possible tokenization methods is denoted as $S(x_i)$. Once the dataset is fixed, the set $S(x_i)$for each word is also fixed, and each method within the set has a certain probability$p(x)$ of being chosen. If we remove some words from the vocabulary, the number of tokenization methods in the set $S(x_i)$ for some words will decrease, reducing the summation term in $\log(*)$ and thereby increasing the overall loss. Therefore, the Unigram algorithm selects 10% to 20% of the words from the vocabulary that contribute the least to the loss increase for deletion in each iteration.

We use the Unigram methods to train our SentencePiece Model on the given dataset. The dataset is then split into training, testing, and validation sets in a ratio of 8:1:1 to obtain the dataset required for training EAPT.

3.2. Pre-training

3.2.1. Disentangled attention

Our model employs a disentangled attention mechanism during the pre-training phase, distinguishing it from traditional BERT models. In BERT, token content embeddings are typically combined with position embeddings, which amalgamates content and positional information, failing to effectively differentiate their distinct contributions to the attention mechanism. To address this issue, we utilize a disentangled attention mechanism, where in each token is represented as two independent vectors, allowing us to encode the content and relative position of the traffic data separately. Additionally, the attention weights between traffic flows are computed using disentangled matrices based on content and relative position, thereby enhancing the model's ability to capture the temporal and structural relationships between packets in the traffic data.

For a token at position i in a BURST sequence, we represent it using two vectors, H_i and $P_{i|j}$, where the former represents the content at position i and the latter represents the relative positional relationship to position$j$. The formula for calculating the cross-attention between tokens i and j is as follows:

$$\begin{aligned} A_{i|j} &= \{H_i, P_{i|j}\} \times \{H_j, P_{j|i}\}^T \\ &= H_i H_j^T + H_i P_{j|i}^T + P_{i|j} H_j^T + P_{i|j} P_{j|i}^T \end{aligned} \tag{2}$$

The attention weights for tokens are composed of four parts: content-to-content, content-to-position, position-to-content, and position-to-position. Since we use relative position embeddings, the position-to-position component is not meaningful and will be ignored in subsequent calculations.

In encrypted traffic, dependencies between packets may span long temporal distances. Disentangled attention mechanisms may struggle to effectively model such long-range dependencies. Therefore, we introduce the concept of relative position to alleviate this issue and improve the modeling capability for complex dependencies. We set n as the maximum relative distance and define the relative distance from token i to token j as $\varphi(i,j) \in [0, 2n)$, as shown in Eq. (3):

$$\varphi(i,j) = \begin{cases} 0 & \text{for } i - j \le -n, \\ 2n-1 & \text{for } i - j \ge n, \\ i - j + n & \text{others} \end{cases} \tag{3}$$

In the computations, we project the content information and positional information of the traffic separately to generate the query, key, and value vectors for content, as well as the projection vectors for relative position. The specific formula for the self-attention mechanism is as follows, where the matrix H represents the input hidden content vectors, H_o is the output vector after applying the disentangled attention mechanism, and Q_c, K_c, V_c are the content vectors generated by projections $W_{q,c}, W_{k,c}, W_{v,c}$ respectively. Additionally, $P \in R^{2n \times d}$ represents the relative position embedding vectors, while $W_{q,r}, W_{k,r}$ are the projection matrices used to map the relative position vectors into the query and key spaces. Q_r and K_r denote the query and key vectors generated from the projected relative position vectors.

$$\begin{aligned} &Q_c = HW_{q,c}, \qquad K_c = HW_{k,c}, \qquad V_c = HW_{v,c}, \\ &Q_r = PW_{q,r}, \qquad K_r = PW_{k,r}. \end{aligned} \tag{4}$$

$$\tilde{A}_{i,j} = \underbrace{Q_i^c K_j^{cT}}_{content-to-content} + \underbrace{Q_i^c K_{\varphi(i,j)}^c}_{content-to-position} + \underbrace{K_j^c Q_{\varphi(j,i)}^r}_{position-to-content}, \tag{5}$$

where $\tilde{A}_{i,j}$ denotes the attention map.

The calculation of $\tilde{A}_{i,j}$ is divided into three parts, corresponding to the three components in Eq. (2). This represents the attention score from token i to token j. When computing the position-to-content component, we use $Q^r_{\varphi(j,i)}$ instead of $Q^r_{\varphi(i,j)}$, because at position i, the attention weight from the position to the content is determined by the key content at j relative to the query position at i. Thus, $Q^r_{\varphi(j,i)}$ is chosen.

$$H_o = softmax\left(\frac{\tilde{A}}{\sqrt{3d}}\right) V_c. \tag{6}$$

Our model generates a large number of parameters. Without appropriate regularization techniques, it may face issues such as overfitting or unstable training. To address this, we apply a scaling factor of $\frac{1}{\sqrt{3d}}$ to the weights of $\tilde{A}$ to control their magnitude, where d represents the size of the weight matrix $\tilde{A}$. This scaling factor helps stabilize the training process and prevents the weights from becoming excessively large.

3.2.2. Replaced burst detection

In the pre-training stage, we employ the Replaced Burst Detection (RBD) task, which involves training two neural networks: a Burst Generator Model and a Burst Discriminator Model . This approach enhances model training efficiency and sample utilization.

The Burst Generator Model (BGM) operates on a given sequence $x = [x_1, x_2, x_3 \dots x_n]$. BGM randomly selects 15% of the tokens for masking, replacing the chosen tokens with [MASK] tokens. The positions of the replaced tokens are denoted as $m = [m_1, \dots, m_i], i \in [1, n]$. We define the sequence with replaced tokens as $x^{\text{masked}} = \text{replace}(x, m, [MASK])$. In the BGM, the sequence x is mapped to a vector representation $h(x) = [h_1, \dots, h_n]$. The probability of generating a specific token at position i is given by the following Eq. (7):

$$p_G(x_i|\boldsymbol{x}) = \exp\left(e(x_i)^T h_G(\boldsymbol{x})_i\right) / \sum_{x'} \exp\left(e(x')^T h_G(\boldsymbol{x})_i\right) \tag{7}$$

where $e(x)$ represents the token embedding of x. The BGM generates a fake token $\bar{x}_i \sim p_G(x_i|x^{masked})$, and finally, the masked tokens are replaced, resulting in $x^{corrupt} = replace(x, m, \bar{x})$. The loss function for the BGM is as follows:

$$\mathcal{L}_{BGM}(x, \theta_G) = \mathbb{E}\left(\sum_{i \in m} -\log p_G(x_i|x^{masked})\right). \tag{8}$$

The Burst Discriminator Model (BDM) is trained to determine whether the tokens in the data are generated or original. For position i, the BDM predicts whether the token x_i is from x^{corrupt} using the following output function:

$$D(x, i) = sigmoid(\omega^T h_D(x)_i), \tag{9}$$

where $h_D(x)_i$ represents the context encoding vector at position i for the input sequence x as determined by the BDM. The parameter vector ω^T is trained to map the encoding vectors into a binary classification probability space. The loss function utilized by BDM is the binary cross-entropy loss, which computes the loss for both the original tokens and the replaced tokens. The loss function is defined as follows:

$$\begin{aligned}\mathcal{L}_{BDM}(x, \theta_D) = \mathbb{E}\Bigg(\sum_{i=1}^{n} -\mathbb{I}\left(x_i^{corrupt} = x_i\right) \log D(x^{corrupt}, i) \\ - \mathbb{I}(x_i^{corrupt} \neq x_i) \log\left(1 - D(x^{corrupt}, i)\right)\Bigg),\end{aligned} \tag{10}$$

where $\mathbb{I}(\cdot)$ represents the indicator function. In the RBD task, $\mathcal{L}_{BGM}$ and $\mathcal{L}_{BDM}$ are optimized together. The final loss function for RBD is given by:

$$\mathcal{L}_{\text{RBD}} = \mathcal{L}_{\text{BGM}} + \lambda \mathcal{L}_{\text{BDM}}, \tag{11}$$

where hyperparameter λ serves as a scaling factor for $\mathcal{L}_{BDM}$, aiming to amplify the loss of BDM. This combined loss function ensures that both the generator and discriminator are optimized together, improving the overall performance of the model in generating and identifying realistic token sequences.

3.3. Fine-tuning

During the fine-tuning phase, we prepared multiple datasets to validate the model's generalization ability across different scenarios. For the tokenization required by downstream tasks, we used the same SentencePiece model (SPM) employed in the pre-training phase. This model maps raw text into input ID sequences and appends classification label IDs. This approach ensures a consistent input format for the model across different datasets.

Subsequently, we added a classifier head to the encoder of the EAPT, which includes a standard softmax classifier and a linear transformation layer. The classifier head maps the encoder's output representations to logits corresponding to the number of classification categories, enabling classification predictions.

Moreover, to validate the real-time performance during the fine-tuning process, we implemented a periodic evaluation mechanism. Specifically, we conducted evaluations at regular intervals (every few steps) to monitor the training status in real-time. We recorded the loss values at these intervals and the results of each evaluation, including calculated evaluation metrics. This data is used to compare the proposed model with other state-of-the-art models to assess its performance.

Through these methods, we can comprehensively evaluate the model's performance on different datasets and tasks, ensuring it has strong generalization capabilities and practical application value.

4. Experiments

4.1. Experiment setup

4.1.1. Datasets

In this experiment, we use four datasets. First, the raw PCAP files are split into individual flows and then converted into hexadecimal format.

Table 2
The statistical details of the datasets.

Dataset	Flow	Packet	Label
USTC-TFC2016	9853	97 115	20
ISCX-VPN-App	2329	77 163	17
ISCX-VPN-Service	3694	60 000	12
CSTNET-TLS1.3	46 372	581 709	120
CIC IoT Dataset 2023	4000	40 000	34

We filter out flows and packets that are too short, and then randomly select flows and packets from each category within each dataset to construct our fine-tuning dataset. The specific details are shown in Table 2.

1. USTC-TFC2016 Dataset [38]: This dataset includes encrypted traffic data from both malicious and benign applications, comprising ten types of benign traffic and ten types of malicious traffic. The malicious traffic data was collected by CTU researchers from real network environments between 2011 and 2015 [39], while the benign traffic data consists of ten types of normal traffic simulated by IXIA BPS.
2. ISCX VPN-nonVPN Dataset [29]: This dataset was created and maintained by the Information Systems Security Lab at Dalhousie University in Canada. It contains labeled data for both virtual private network (VPN) and non-VPN traffic. For this study, only the VPN portion was used, which was subdivided into the ISCX-VPN-Service dataset comprising 12 categories and the ISCX-VPN-App dataset covering 17 applications for experimental testing.
3. CSTNET-TLS1.3 Dataset [14]: This dataset was collected by the Institute of Information Engineering, Chinese Academy of Sciences, from March to July 2021 on CSTNET. It includes 120 applications, adopting the latest encryption protocols and providing a more diverse and recent set of applications compared to the other datasets. This allows for a comprehensive evaluation of the model's recognition capabilities.
4. CIC IoT Dataset 2023 [40]: This dataset, published by the Canadian Institute for Cybersecurity (CIC), aims to facilitate security research in IoT environments. It simulated the network behavior of various IoT devices, encompassing both normal traffic and multiple types of attack traffic, such as Denial of Service (DoS) attacks and malware propagation. This enabled researchers to comprehensively analyze potential threats and anomalies in IoT environments. We further divided it into datasets containing 8 categories and 2 categories.

By utilizing these three datasets, we can validate the model's generalization ability and practical application performance across different scenarios.

4.1.2. Implementation details

We utilized the CSTNET-TLS1.3 dataset for pre-training. First, we processed the entire dataset and trained a SentencePiece model to generate a vocabulary of size 128,100, where 128,000 entries were selected vocabulary and the remaining 100 were reserved tokens for future modifications. Subsequently, we used this vocabulary to tokenize the pre-training dataset. During pre-training, we set the hyperparameter λ to 50, as detailed in Experiment 4.6. The pre-training was configured for 230,000 steps, with a checkpoint saved every 10,000 steps. The learning rate was set to 5e−5, with a training batch size of 32 and an evaluation batch size of 16.

In the fine-tuning stage, our model requires only a small amount of labeled data for training. Therefore, we select at most 500 flows and 5000 packets from each category within each dataset. All the data was then split into training, validation, and test sets in a ratio of 8:1:1. The fine-tuning parameters were set as follows: both the training and

Table 3
Comparison results on USTC-TFC, ISCX-Tor and CSTNET-TLS 1.3 datasets.

Dataset	USTC-TFC(%)				ISCX-VPN-app(%)				ISCX-VPN-service(%)				CSTNET-TLS1.3(%)			
Method	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1
FlowPrint	81.46	643.4	70.02	65.73	85.63	65.96	65.61	64.91	79.62	80.42	78.12	78.20	12.41	12.44	13.42	10.96
AppScanner	89.54	89.84	89.68	88.92	63.67	47.54	50.68	48.95	71.82	73.39	72.25	71.97	67.12	62.46	61.28	61.03
CUMUL	56.75	61.71	57.38	55.13	53.65	41.29	45.35	42.36	56.10	58.83	56.76	56.68	53.91	49.42	50.60	49.04
BIND	84.57	86.81	83.82	83.96	67.67	51.52	51.53	49.65	75.34	75.83	74.88	74.20	79.64	76.05	76.50	75.60
K-fp	–	–	–	–	60.70	54.78	54.30	53.03	64.30	64.92	64.17	63.95	40.36	39.69	40.44	39.02
DF	77.87	78.83	78.19	75.93	63.26	56.96	46.92	46.93	71.54	71.92	71.04	71.02	78.39	76.91	74.92	76.15
FS-Net	88.46	88.46	89.20	88.40	65.96	47.93	48.59	48.37	72.05	75.02	72.38	71.31	85.93	83.92	82.99	82.93
Deeppacket	96.40	96.50	96.31	96.41	96.97	97.48	96.97	96.85	93.29	93.77	93.06	93.21	81.23	42.64	25.79	41.13
TSCRNN	98.70	98.70	98.70	98.70	92.70	92.60	92.60	91.70	91.50	91.50	91.44	91.44	–	–	–	–
GraphDApp	87.89	82.26	82.60	82.34	63.28	59.00	54.72	55.58	59.77	60.45	62.20	60.36	70.34	64.64	65.10	64.40
PERT	99.09	99.11	99.10	99.11	82.29	70.92	71.73	69.92	93.52	94.00	93.49	93.68	89.15	88.46	87.19	87.41
ET-BERT	99.15	99.15	99.16	99.16	99.62	**99.36**	99.38	99.37	98.90	98.91	98.90	98.90	**97.37**	**97.42**	**97.42**	**97.41**
YaTC	97.86	97.85	97.86	97.86	98.57	98.57	98.47	98.47	98.07	98.07	98.04	98.04	–	–	–	–
EAPT	**99.15**	**99.18**	**99.16**	**99.16**	**99.62**	99.27	**99.48**	**99.37**	**98.97**	**98.98**	**98.97**	**98.97**	97.14	97.16	97.16	97.15

validation batch sizes were set to 32, and the learning rate was set to 2e−5. Additionally, validation was performed every 1000 steps to monitor the training performance in real time.

4.1.3. Evaluation metrics

Our fine-tuned model involves multi-class classification tasks, thus we selected a series of evaluation metrics, including Accuracy, Precision, Recall, and F1-Score, to comprehensively assess the model's performance. These metrics facilitate the subsequent comparative evaluation of our model against other models.

- Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}, \tag{12}$$

- Precision

$$\text{Precision} = \frac{TP}{TP + FP}, \tag{13}$$

- Recall

$$\text{Recall} = \frac{TP}{TP + FN}, \tag{14}$$

- F1-Score

$$\text{F1-Score} = \frac{2\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \tag{15}$$

where TP (True Positive), the number of samples that the model correctly predicted as positive;TN (True Negative), the number of samples that the model correctly predicted as negative;FP (False Positive), the number of samples that the model incorrectly predicted as positive;FN (False Negative), the number of samples that the model incorrectly predicted as negative.

Furthermore, because the datasets used during the fine-tuning process are multi-class, we cannot directly calculate Precision, Recall, and F1-Score using the aforementioned formulas. Instead, we need to average these metrics. Here, we employ Macro-averaging [5] to process the data. This involves first calculating the metrics for each class individually, and then computing the arithmetic mean across all classes. The specific calculation methods are as follows:

$$Macro\_Precision = \frac{1}{n}\sum_{i=1}^{n} Precision_i. \tag{16}$$

$$Macro\_Recall = \frac{1}{n}\sum_{i=1}^{n} Recall_i. \tag{17}$$

$$Macro\_F1 = \frac{1}{n}\sum_{i=1}^{n} F1_i. \tag{18}$$

4.2. Comparison experiments

To thoroughly assess the performance of the proposed model, we employed the four key evaluative metrics previously mentioned–Accuracy, Precision, Recall, and F1-Score–as benchmarks for an exhaustive comparative analysis with other approaches from existing research. These approaches include plaintext feature-based fingerprinting methods: FlowPrint [41] and traditional statistical feature-based machine learning methods: AppScanner [42], CUMUL [43], BIND [44], and k-fingerprinting (K-fp) [45]; deep learning-based methods: Deep Fingerprinting (DF) [28], FS-Net [29], DeepPacket [46], TSCRNN [47]; graph neural network-based method: GraphDApp [30]; and pre-training methods: PERT [33], ET-BERT [14] and YaTC [48]. The comparison results are shown in Table 3:

Based on the comparison table, our method demonstrates robust performance across all datasets, even with a limited amount of labeled data used for fine-tuning. Based on the comparison table, it outperforms other methods on the ISCX-VPN-service dataset across all four metrics. On the USTC-TFC dataset, our model achieves the highest accuracy and matches the ET-BERT model on the other three metrics. On the ISCX-VPN-app dataset, our method surpasses most other approaches, achieving lower precision compared to ET-BERT but higher recall, resulting in comparable accuracy and F1 scores. While on the CSTNET-TLS1.3 dataset our method performs slightly below ET-BERT due to differences in pre-training data and model size, it still maintains strong performance relative to other methods. The specific reasons for these performance differences will be discussed in subsequent experiments.

4.3. Model generalization analysis:

To further evaluate the model's generalization ability, we tested it on the updated CIC IoT Dataset 2023. For this purpose, we designed two tasks: Task 1 involves IoT attack detection with two categories, and Task 2 involves IoT attack method detection with eight categories. The experimental results are presented in Table 4. Comparative analysis indicates that, after fine-tuning with a small amount of labeled data, our model continues to demonstrate superior performance on the new dataset, thereby confirming its robust generalization capability.

4.4. Model approach analysis

To demonstrate that our model can better understand the contextual relationships within encrypted traffic data, we conducted a comparative analysis of the accuracy over the first 10 epochs of training. As shown in the Fig. 2, our model exhibits higher accuracy right from the initial stages. This indicates that our tokenization method and the relative position-aware pre-training task (RBD task) are more effective

Table 4
Model performance on CIC IoT dataset 2023.

Method	Task 1(%)		Task 2(%)	
	AC	F1	AC	F1
FSNet	83.57	45.58	48.83	42.50
DeepPacket	90.15	76.73	75.00	57.36
TSCRNN	89.68	63.73	75.78	60.67
YaTC	92.47	82.65	83.94	76.73
ET-Bert	94.90	93.22	92.12	91.88
EAPT	**95.60**	**95.60**	**93.63**	**93.60**

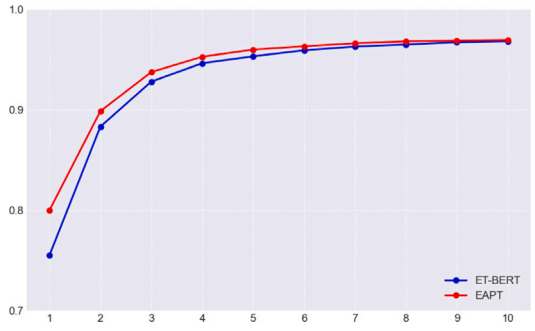


Fig. 2. Comparative accuracy analysis over initial training epochs.

Table 5
Comparison of different models.

Model	FLOPs (M)	Parameters (M)
DF	2.8e+0	9.3e−1
FS-Net	1.0e+1	3.2e+0
GraphDapp	3.8e−2	1.1e−2
PERT	3.9e+3	3.1e+1
ET-BERT	1.1e+4	1.1e+2
EAPT	2.6e+3	2.2e+1

in capturing the temporal dependencies and structural features between packets in encrypted traffic data.

Compared to other methods, our model also demonstrates faster convergence. This suggests that by independently modeling content and positional information and integrating them into the attention mechanism, our model can more efficiently learn the critical features required for encrypted traffic recognition. Our approach not only enhances initial recognition capabilities but also accelerates the overall convergence process.

In summary, our experimental results fully demonstrate that the tokenization method and the disentangled attention mechanism employed by our EAPT model effectively enhance the model's ability to understand the semantics and structure of encrypted traffic data. Consequently, this leads to superior performance in encrypted traffic recognition tasks. This provides a valuable technical approach for further optimizing encrypted traffic analysis models.

4.5. Model complexity analysis

To comprehensively evaluate the trade-off between model complexity and performance, we conducted an in-depth analysis using two metrics: floating-point operations per second (FLOPs) and model parameter count. As shown in the Table 5, our model achieved outstanding performance across all datasets while maintaining controlled model complexity.

Additionally, in Table 6, we compare the differences between the EAPT model and the ET-BERT model across multiple dimensions,

Table 6
Comparison of model parameters and pre-training details.

Metric	ET-BERT	EAPT
Parameters (M)	110	22
Dataset Size (G)	30	4
Steps (W)	50	23
Model Size (M)	114	682

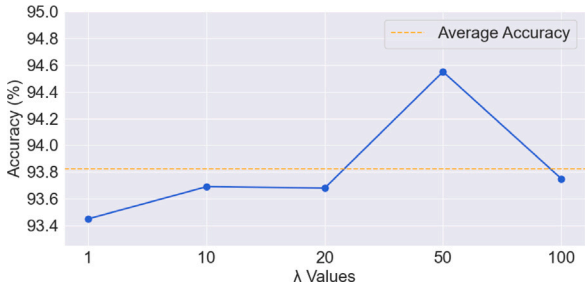


Fig. 3. Accuracy vs. λ values.

including parameter size, pre-training dataset size, number of pre-training steps, and size of the pre-trained model. The comparison shows that EAPT significantly outperforms ET-BERT in terms of model scale and training resources.

First, EAPT has a parameter count of 22M, which is only about one-fifth of ET-BERT's, greatly reducing the computational complexity of the model. Second, the pre-training dataset used by EAPT is 4 GB, much smaller than ET-BERT's 30 GB, indicating that EAPT is more efficient in terms of data requirements. Additionally, EAPT's number of pre-training steps is 230,000, which is more than half less than ET-BERT's 500,000 steps, significantly shortening the training time cost. Finally, the size of EAPT's pre-trained model is only 114 MB, compared to ET-BERT's 682 MB, demonstrating our model's advantages in storage and deployment.

These improvements make EAPT more practical for use in resource-constrained environments while maintaining exceptional performance in encrypted traffic classification.

4.6. Macrobenchmarking

To determine the size of the hyperparameter λ in Eq. (11), we referenced the paper [32] and experimented with various hyperparameter settings, as shown in Fig. 3. It is evident that when $\lambda = 50$, EAPT achieves the best performance on the RBD detection task. This is because BDM is a binary classification task (determining whether a token has been replaced), resulting in a smaller loss, whereas BGM has a relatively larger loss due to the need to select an appropriate token for replacement from a large vocabulary, akin to a multi-class classification task. The core idea of our RBD task is to achieve efficient pre-training through BDM. By amplifying the loss from BDM, we enhance the performance of the RBD task.

4.7. Evaluating header-only training

In this experiment, we investigate the effect of training the model using only packet header information to verify the necessity of learning from the complete packet. By training the model in the pre-training stage with either the full packet or packet header data, we aim to reveal the limitations of header-only features and underscore the importance of comprehensive packet content analysis. Under identical experimental settings and evaluation criteria, we compare the classification performance of the two training methods. The results are shown in the Table 7:

Table 7
Comparison of Header-Only and Full Packet training on different datasets.

Dataset	Header-Only training (%)	Full Packet training (%)
USTC-TFC	98.07	99.15 (1.08 ↑)
ISCX-VPN-app	98.38	99.62 (1.24 ↑)
ISCX-VPN-service	97.57	98.97 (1.40 ↑)
CSTNET-TLS1.3	94.07	97.14 (3.07 ↑)

The experimental results indicate that the model trained solely on headers performs significantly lower in classification accuracy than the model trained on full packets. This outcome demonstrates that while headers provide essential flow control information, they lack sufficient contextual data to support complex traffic classification tasks.

5. Conclusion

We propose an encrypted traffic classification model via adversarial pre-trained transformers-EAPT, to address key challenges in encrypted traffic identification. By utilizing SentencePiece technology for tokenizing encrypted traffic data, we effectively resolve the issue of overly large token granularity. During the pre-training phase, the model introduces a disentangled attention mechanism and a Replaced BURST detection task, enabling better capture of temporal and structural relationships between packets. These improvements also accelerate the pre-training process, reduce the number of model parameters, and enhance generalization capabilities. Experimental results show that the EAPT model performs exceptionally well across multiple datasets, significantly exceeding existing methods in metrics such as accuracy, recall, and F1 score, while greatly reducing the required amount of pre-training data and model complexity.

Our model classifies traffic by learning the dependencies within the packet context. In this process, we recognize that, in addition to packet length, external features such as temporal information and packet direction may also lead to privacy leakage. Although existing techniques like traffic obfuscation and packet padding can partially mitigate these issues, they also introduce new challenges for encrypted traffic identification. Future work will further investigate the impact of these privacy protection technologies on classification models and strive to enhance the model's effectiveness in a broader range of practical application scenarios.

CRediT authorship contribution statement

Mingming Zhan: Writing – original draft, Software, Methodology, Conceptualization. **Jin Yang:** Writing – review & editing, Software, Project administration. **Dongqing Jia:** Writing – review & editing, Visualization, Supervision, Investigation. **Geyuan Fu:** Resources, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The work is supported by the National Natural Science Foundation of China (No. 62162057, No. 61872254), the Sichuan Science and Technology Program, China (2021JDRC0004), and the Key Laboratory of Data Protection and Intelligent Management of the Ministry of Education, China (SCUSAKFKT202402Y).

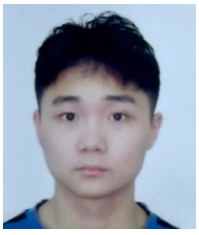
Data availability

Data will be made available on request.

References

[1] W. Dong, J. Yu, X. Lin, G. Gou, G. Xiong, Deep learning and pre-training technology for encrypted traffic classification: A comprehensive review, Neurocomputing (2024) 128444.
[2] C. Dong, C. Zhang, Z. Lu, B. Liu, B. Jiang, CETAnalytics: Comprehensive effective traffic information analytics for encrypted traffic classification, Comput. Netw. 176 (2020) 107258.
[3] Y. Ma, Z. Li, H. Xue, J. Chang, A balanced supervised contrastive learning-based method for encrypted network traffic classification, Comput. Secur. 145 (2024) 104023.
[4] R. Sharma, S. Dangi, P. Mishra, A comprehensive review on encryption based open source cyber security tools, in: 2021 6th International Conference on Signal Processing, Computing and Control, IEEE, 2021, pp. 614–619.
[5] S. Sengupta, N. Ganguly, P. De, S. Chakraborty, Exploiting diversity in android TLS implementations for mobile app traffic classification, in: The World Wide Web Conference, 2019.
[6] N. Malekghaini, E. Akbari, M.A. Salahuddin, N. Limam, R. Boutaba, B. Mathieu, S. Moteau, S. Tuffin, Deep learning for encrypted traffic classification in the face of data drift: An empirical study, Comput. Netw. 225 (2023) 109648.
[7] S. Izadi, M. Ahmadi, R. Nikbazm, Network traffic classification using convolutional neural network and ant-lion optimization, Comput. Electr. Eng. 101 (2022) 108024.
[8] X. Yan, L. He, Y. Xu, J. Cao, L. Wang, G. Xie, High-speed encrypted traffic classification by using payload features, Digit. Commun. Netw. (2024).
[9] Y. Li, X. Chen, W. Tang, Y. Zhu, Z. Han, Y. Yue, Interaction matters: Encrypted traffic classification via status-based interactive behavior graph, Appl. Soft Comput. 155 (2024) 111423.
[10] W. Wang, H. Lee, Y. Huang, E. Bertino, N. Li, Towards efficient privacy-preserving deep packet inspection, in: European Symposium on Research in Computer Security, Springer, 2023, pp. 166–192.
[11] A. Azab, M. Khasawneh, S. Alrabaee, K.-K.R. Choo, M. Sarsour, Network traffic classification: Techniques, datasets, and challenges, Digit. Commun. Netw. (2022).
[12] Y. Liu, X. Wang, B. Qu, F. Zhao, ATVITSC: A novel encrypted traffic classification method based on deep learning, IEEE Trans. Inf. Forensics Secur. 19 (2024) 9374–9389.
[13] J. Dai, X. Xu, F. Xiao, GLADS: A global-local attention data selection model for multimodal multitask encrypted traffic classification of IoT, Comput. Netw. 225 (2023) 109652.
[14] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, J. Yu, Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification, in: ACM Web Conference 2022, 2022, pp. 633–642.
[15] R.T. Elmaghraby, N.M. Abdel Aziem, M.A. Sobh, A.M. Bahaa-Eldin, Encrypted network traffic classification based on machine learning, Ain Shams Eng. J. 15 (2) (2024) 102361.
[16] M.V. Kotpalliwar, R. Wajgi, Classification of attacks using support vector machine (svm) on kddcup'99 ids database, in: 2015 Fifth International Conference on Communication Systems and Network Technologies, IEEE, 2015, pp. 987–990.
[17] A. Chandrasekhar, K. Raghuveer, Confederation of fcm clustering, ann and svm techniques to implement hybrid nids using corrected kdd cup 99 dataset, in: 2014 International Conference on Communication and Signal Processing, IEEE, 2014, pp. 672–676.
[18] A.H. Lashkari, G.D. Gil, M.S.I. Mamun, A.A. Ghorbani, Characterization of tor traffic using time based features, in: International Conference on Information Systems Security and Privacy, vol. 2, SciTePress, 2017, pp. 253–262.
[19] J. Li, H. Zhou, S. Wu, X. Luo, T. Wang, X. Zhan, X. Ma, FOAP: Fine-grained Open-World android app fingerprinting, in: 31st USENIX Security Symposium, 2022, pp. 1579–1596.
[20] C. Fu, et al., Detecting unknown encrypted malicious traffic in real time via flow interaction graph analysis, in: Network and Distributed System Security Symposium, ISOC, 2023.
[21] M. Garouani, A. Ahmad, M. Bouneffa, M. Hamlich, AMLBID: an auto-explained automated machine learning tool for big industrial data, SoftwareX 17 (2022) 100919.
[22] G. Xie, Q. Li, Y. Jiang, Self-attentive deep learning method for online traffic classification and its interpretability, Comput. Netw. 196 (2021) 108267.
[23] H. Zhang, L. Yu, X. Xiao, Q. Li, F. Mercaldo, X. Luo, Q. Liu, Tfe-gnn: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification, in: Proceedings of the ACM Web Conference 2023, 2023, pp. 2066–2075.
[24] Z. Okonkwo, E. Foo, Q. Li, Z. Hou, A CNN based encrypted network traffic classifier, in: 2022 Australasian Computer Science Week, ACSW '22, 2022, pp. 74–83.

[25] X. Yuan, C. Li, X. Li, DeepDefense: identifying ddos attack via deep learning, in: 2017 IEEE International Conference on Smart Computing, IEEE, 2017, pp. 1–8.
[26] S. Cui, B. Jiang, Z. Cai, Z. Lu, S. Liu, J. Liu, A session-packets-based encrypted traffic classification using capsule neural networks, in: 2019 IEEE 21st International Conference on High Performance Computing and Communications; IEEE 17th International Conference on Smart City; IEEE 5th International Conference on Data Science and Systems, IEEE, 2019, pp. 429–436.
[27] T. Shapira, Y. Shavitt, FlowPic: A generic representation for encrypted traffic classification and applications identification, IEEE Trans. Netw. Serv. Manag. 18 (2) (2021) 1218–1232.
[28] P. Sirinam, M. Imani, M. Juarez, M. Wright, Deep fingerprinting: Undermining website fingerprinting defenses with deep learning, in: 2018 ACM SIGSAC Conference on Computer and Communications Security, 2018, pp. 1928–1943.
[29] C. Liu, L. He, G. Xiong, Z. Cao, Z. Li, Fs-net: A flow sequence network for encrypted traffic classification, in: IEEE INFOCOM 2019-IEEE Conference on Computer Communications, IEEE, 2019, pp. 1171–1179.
[30] M. Shen, J. Zhang, L. Zhu, K. Xu, X. Du, Accurate decentralized application identification via encrypted traffic analysis using graph neural networks, IEEE Trans. Inf. Forensics Secur. 16 (2021) 2367–2380.
[31] P. He, J. Gao, W. Chen, Debertav3: Improving deberta using electra-style pre-training with gradient-disentangled embedding sharing, 2021, arXiv preprint arXiv:2111.09543.
[32] K. Clark, M.-T. Luong, Q.V. Le, C.D. Manning, Electra: Pre-training text encoders as discriminators rather than generators, 2020, arXiv preprint arXiv:2003.10555.
[33] H.Y. He, Z.G. Yang, X.N. Chen, PERT: Payload encoding representation from transformer for encrypted traffic classification, in: 2020 ITU Kaleidoscope: Industry-Driven Digital Transformation, IEEE, 2020, pp. 1–8.
[34] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, A. Hotho, A survey of network-based intrusion detection data sets, Comput. Secur. 86 (2019) 147–167.
[35] R. Boutaba, M.A. Salahuddin, N. Limam, S. Ayoubi, N. Shahriar, F. Estrada-Solano, O.M. Caicedo, A comprehensive survey on machine learning for networking: evolution, applications and research opportunities, J. Internet Serv. Appl. 9 (1) (2018) 1–99.
[36] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, Bert: Pre-training of deep bidirectional transformers for language understanding, 2018, arXiv preprint arXiv:1810.04805.
[37] H. Wang, J. Li, H. Wu, E. Hovy, Y. Sun, Pre-trained language models and their applications, Engineering 25 (2023) 51–65.
[38] W. Wang, M. Zhu, J. Wang, X. Zeng, Z. Yang, End-to-end encrypted traffic classification with one-dimensional convolution neural networks, in: 2017 IEEE International Conference on Intelligence and Security Informatics, IEEE, 2017, pp. 43–48.
[39] S. Garcia, M. Grill, J. Stiborek, A. Zunino, An empirical comparison of botnet detection methods, Comput. Secur. 45 (2014) 100–123.
[40] E.C.P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, A.A. Ghorbani, CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment, Sensors 23 (13) (2023) 5941.
[41] T. Van Ede, R. Bortolameotti, A. Continella, J. Ren, D.J. Dubois, M. Lindorfer, D. Choffnes, M. Van Steen, A. Peter, Flowprint: Semi-supervised mobile-app fingerprinting on encrypted network traffic, in: Network and Distributed System Security Symposium, vol. 27, 2020.
[42] V.F. Taylor, R. Spolaor, M. Conti, I. Martinovic, Appscanner: Automatic fingerprinting of smartphone apps from encrypted network traffic, in: 2016 IEEE European Symposium on Security and Privacy, IEEE, 2016, pp. 439–454.
[43] A. Panchenko, F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, K. Wehrle, Website fingerprinting at internet scale., in: Network and Distributed System Security Symposium, 2016.
[44] K. Al-Naami, S. Chandra, A. Mustafa, L. Khan, Z. Lin, K. Hamlen, B. Thuraisingham, Adaptive encrypted traffic fingerprinting with bi-directional dependence, in: 32nd Annual Conference on Computer Security Applications, 2016, pp. 177–188.
[45] J. Hayes, G. Danezis, K-fingerprinting: A robust scalable website fingerprinting technique, in: 25th USENIX Security Symposium, 2016, pp. 1187–1203.
[46] M. Lotfollahi, M. Jafari Siavoshani, R. Shirali Hossein Zade, M. Saberian, Deep packet: A novel approach for encrypted traffic classification using deep learning, Soft Comput. 24 (3) (2020) 1999–2012.
[47] K. Lin, X. Xu, H. Gao, TSCRNN: A novel classification scheme of encrypted traffic based on flow spatiotemporal features for efficient management of IIoT, Comput. Netw. 190 (2021) 107974.
[48] R. Zhao, M. Zhan, X. Deng, Y. Wang, Y. Wang, G. Gui, Z. Xue, Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation, in: AAAI Conference on Artificial Intelligence, vol. 37, (4) 2023, pp. 5420–5427.



Mingming Zhan received the BE degree from the Sichuan University, in 2023. Currently, he is working toward the MS degree in the School of Cyber Science and Engineering, Sichuan University, Chengdu, China. His research interests include encrypted traffic classification, natural language processing and Transformer-based models.

Jin Yang received the M.S. and Ph.D. degrees in computer science from Sichuan University, Sichuan, China, in 2004 and 2007 respectively. He is currently an Associate Professor with the School of Cyber Science and Engineering, Sichuan University, China. His main research interests include network security, knowledge discovery, and expert systems.

Dongqing Jia is working toward the Ph.D. degree in the School of Cyber Science and Engineering at Sichuan University, Sichuan, China. Her primary areas of interest and research include intrusion detection, malicious traffic analysis, artificial intelligence, and network security.

Geyuan Fu received the BE degree from the Sichuan University, in 2023. His research interests include encrypted traffic classification,network security and deep learning.